Rag实现相关

RAG_01_读取数据集.py

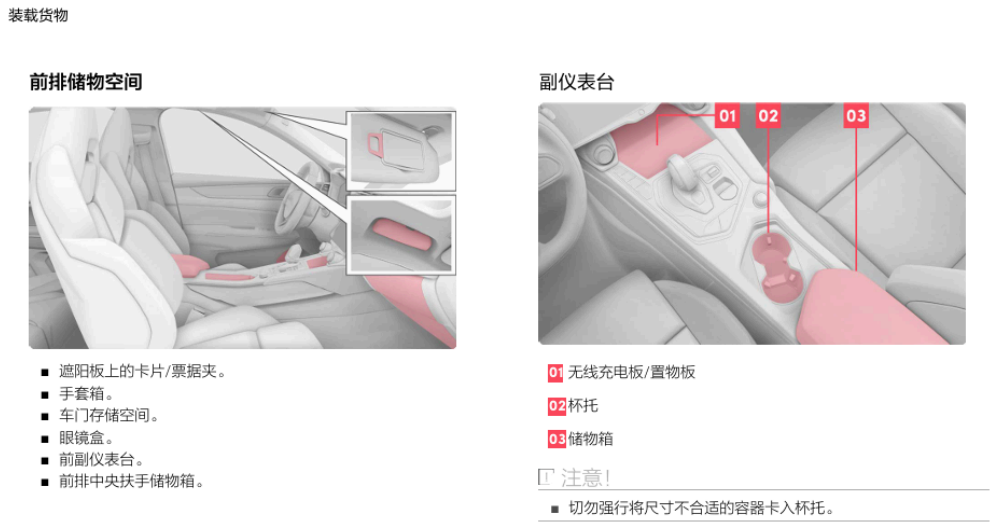
In [4]:
```
import json  # 处理json数据

import pdfplumber  # 解析PDF内容

# 这个库对“能被 PDF 内部文本层表达的文字”提取效果更好；
# 如果 PDF 是扫描版（图片），extract_text() 往往会得到 None 或很少内容，需要 OCR

with open("./Week06/questions.json", "r", encoding="utf-8") as f:
    questions = json.load(f)
# 用questions.json存储一些预定义的问题（针对pdf），用于检验rag效果并进行优化
print(f"load {len(questions)} questions")
print(f"Sample: \n {questions[0]}")
```

```
load 301 questions
Sample:
 {'question': '“前排座椅通风”的相关内容在第几页？', 'answer': '', 'reference':
''}
```

In [5]:
```
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")  # 以这个文件作为模拟的知识库
print(f"PDF页数: \npages: {len(pdf.pages)}")
print(f"pdf第23页解析内容: ")
print(
    pdf.pages[23].extract_text()
)  # extract_text() 通常是“尽量把页面上可抽取的文字拼起来，乱序丢失排版
```

```
PDF页数:
pages: 354
pdf第23页解析内容:
装载货物
前排储物空间 副仪表台
■ 遮阳板上的卡片/票据夹。 01无线充电板/置物板
■ 手套箱。
02杯托
■ 车门存储空间。
■ 眼镜盒。 03储物箱
■ 前副仪表台。
■ 前排中央扶手储物箱。 注意！
■ 切勿强行将尺寸不合适的容器卡入杯托。
24
```

23页内容，所有文字都提取了出来，只是乱序



pdf解析其他方案

参考文章：https://mp.weixin.qq.com/s/4x0orb5JCy1Ga2drRrkjmQ

一、更成熟高效的 PDF 解析路线（少算坐标）

1）Unstructured（RAG 场景最常用的一类）

核心优势：**自动版面分析 + 结构化元素输出**（Title / NarrativeText / ListItem / Table...），对你这种“图文混排+两栏+编号说明”更友好。你不需要手写两栏分割线或 bbox。

示例（直接用）：

```
# pip install "unstructured[pdf]" unstructured
from unstructured.partition.pdf import partition_pdf

elements = partition_pdf(
    filename="./Week06/汽车知识手册.pdf",
    strategy="hi_res",          # 高质量布局解析（更适合复杂排版）
    infer_table_structure=True, # 尝试识别表格结构
    include_page_breaks=False,
)

# elements 是结构化块：每块有 text + 类型 + 元信息（页码等）
for e in elements[:20]:
    print(type(e).__name__, getattr(e, "metadata", None).page_number,
e.text[:80])
```

拿去做 RAG 的典型做法：

- 只保留你关心的类型（Title/ListItem/NarrativeText...）

- 按 `page_number` + `Title` 作为 section 边界来 chunk
- 元数据直接带页码，后续可溯源引用

> 备注：`strategy="hi_res"` 通常更准，但依赖/耗时也更高；如果你追求速度，可先试 `strategy="fast"`，不够再切 `hi_res`。

2）Docling（把 PDF 变成“结构化文档/Markdown”）

这类工具更像“把 PDF 还原成可读文档”，对混排、图注、标题层级往往更强，输出更适合直接入库（Markdown/JSON）。

优点：结构化强、RAG 友好
缺点：环境/依赖可能比 PyMuPDF 重一些

（如果你想走这条，我可以按你环境给你一份最小可跑 demo。）

3）PyMuPDF4LLM / LlamaIndex Loader（轻量、落地快）

这类更偏“把 PDF 尽量变成 LLM 友好的文本”，通常比你手写坐标轻松很多，但遇到非常复杂的混排/表格时，结构化能力可能不如 Unstructured/Docling。

4）商业级（最省心但可能收费）：Adobe PDF Extract / 各类 Document AI

如果你在公司环境能用 API，这种对阅读顺序、表格、标题层级、图注的处理通常最强，基本不需要你再定制。

二、OCR 需要模型吗？笔记本能跑吗？

需要，但一般 **不需要你训练**：

- **PaddleOCR**：自带预训练模型（中文很强），开箱即用。
- **Tesseract**：是传统 OCR 引擎（也算模型/字库），安装后直接用。

笔记本能不能跑：能。

- 只有 CPU 也能跑，只是速度慢一些。
- 页面多（354 页）时，建议：
  - 只对“疑似扫描页”跑 OCR（别全量 OCR）
  - 或者做缓存（把 OCR 结果落盘）

聊天框给你一个“自动判别 + OCR 兜底”的最小可跑示例（推荐）

思路：先用 PyMuPDF 提取文本；如果很短就把页面渲染成图片走 OCR。

方案 A：PaddleOCR（中文推荐）

```
# pip install pymupdf paddleocr
import fitz
from paddleocr import PaddleOCR

ocr = PaddleOCR(use_angle_cls=True, lang="ch")  # 第一次会下载模型（联网）

def extract_page_text(pdf_path, page_index, min_chars=30, dpi=200):
    doc = fitz.open(pdf_path)
    page = doc[page_index]

    # 1) 先走文本层
    txt = page.get_text("text").strip()
    if len(txt) >= min_chars:
        return txt, "text-layer"

    # 2) 再 OCR 兜底
    mat = fitz.Matrix(dpi / 72, dpi / 72)
    pix = page.get_pixmap(matrix=mat)
    img_bytes = pix.tobytes("png")

    result = ocr.ocr(img_bytes, cls=True)
    ocr_lines = []
    for line in result[0]:
        ocr_lines.append(line[1][0])
    return "\n".join(ocr_lines).strip(), "ocr"

text, mode = extract_page_text("./Week06/汽车知识手册.pdf", 23)
print("mode:", mode)
print(text[:1000])
```

方案 B：Tesseract（更轻，但中文效果通常不如 PaddleOCR）

需要你本机装 tesseract + 中文语言包。

我建议你怎么选（结合你这本车主手册）

- **优先试 Unstructured（hi_res）**：最少定制、输出结构化，最适合 RAG 入库。
- 如果 PDF 里混有扫描页：加上“自动判别 + OCR fallback”（只对少数页 OCR）。
- 真要追求极致效果/省心：商业 Document AI。

RAG 02 RAG检索TF-IDF.py

In [26]:
```python
import json

import jieba
import numpy as np
import pdfplumber
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize
```

In [10]:
```python
# 读取问题集与 PDF 知识库（按页组织）
questions = json.load(open("./Week06/questions.json", "r", encoding="utf-8"))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")
pdf_content = []
# 间隔每一页的所有内容当做一个chunk
for page_idx in range(len(pdf.pages)):
    pdf_content.append(
        {
            "page": "page_" + str(page_idx + 1),
            "content": pdf.pages[page_idx].extract_text(),
        }
    )
```

In [11]:
```python
# 分词，将每页内容变为token序列
questions_words = [" ".join(jieba.lcut(x["question"])) for x in questions]
pdf_words = [" ".join(jieba.lcut(x["content"])) for x in pdf_content]
```

```
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\86177\AppData\Local\Temp\jieba.cache
Loading model cost 0.557 seconds.
Prefix dict has been built successfully.
```

In [13]:
```python
print(questions_words[0], "\n", pdf_words[0])
```

```
“ 前排 座椅 通风 ” 的 相关 内容 在 第几页 ？
 欢迎
 感谢您 选择 了 具有 优良 安全性 、 舒适性 、 动力性 和 经济性 的 Lynk & Co 领克 汽车 。
 首次 使用 前 请 仔细 、 完整 地 阅读 本手册 内容 ， 将 有助于 您 更好 地 了解 和 使用 车辆 。
 本手册 中 的 所有 资料 均 为 出版 时 的 最新 资料 ， 但本 公司 将 对 产品 进行 不断 的 改进 和 优化 ， 您 所购 的 车辆 可能 与 本手册 中 的 描述 有所不同 ， 请以 实际
 接收 的 车辆 为准 。
 如 您 有 任何 问题 ， 或 需要 预约 服务 ， 请拨 打电话 4006 - 010101 联系 我们 。 您 也 可以 开车 前往 Lynk & Co 领克 中心 。
 在 抵达 之前 ， 请 您 注意 驾车 安全 。
 © 领克 汽车 销售 有限公司
```

In [18]:
```python
# 构建tf-idf词空间并向量化
tfidf = TfidfVectorizer()
tfidf.fit(questions_words + pdf_words)
questions_embedding = tfidf.transform(questions_words)
pdf_embedding = tfidf.transform(pdf_words)
```

In [43]:
```python
# 向量归一化：让点积=余弦相似度
questions_embedding = normalize(questions_embedding)
pdf_embedding = normalize(pdf_embedding)
questions_top10 = questions  # 储存top10结果，原questions储存top1结果
```

In [40]:
```python
# 可一验证每一个向量的长度都为1
from scipy.sparse.linalg import norm

vec = questions_embedding[10]  # 1 x vocab_size 的稀疏向量
print(type(vec))  # 稀疏矩阵
print(vec[0])  # 只记录那个位置有值
print(vec.toarray()[0])  # 通过toarray()转换为向量
length = norm(vec)  # 默认就是 L2 范数
print(length)
```

```
<class 'scipy.sparse._csr.csr_matrix'>
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 8 stored elements and shape (1, 4568)>
  Coords        Values
  (0, 1047)     0.41401179368067326
  (0, 1234)     0.4773286596063898
  (0, 1508)     0.1756378540171288
  (0, 1637)     0.2656083215394678
  (0, 3453)     0.39523157163055683
  (0, 3772)     0.3159547425018198
  (0, 3909)     0.3219361731498101
  (0, 3984)     0.3737336686353367
[0. 0. 0. ... 0. 0. 0.]
1.0
```

In [46]:
```python
# 检索 Top1：每个问题找最相关的1页
for question_idx, question in enumerate(questions_embedding):
    # [1,pdf页数] 每一列都是question与该页的余弦相似度结果
    score = question @ pdf_embedding.T
    # 将稀疏矩阵转换为向量
    score = score.toarray()[0]  # [pdf页数]
    # 取最大值对应的索引
    max_score_idx = np.argmax(score)
    # 将结果保存到json数据
    questions[question_idx]["reference"] = "page_" + str(max_score_idx)
with open("./Week06/tfidf_retrival_top1.json", "w", encoding="utf-8") as f:
    json.dump(questions, f, ensure_ascii=False, indent=4)
```

In [48]:
```python
# 检索Top10:每个问题最相关的10页
for question_idx, question in enumerate(questions_embedding):
    score = question @ pdf_embedding.T
    score = score.toarray()[0]
    # argsort()从小到大排列；::-1:翻转，+1变为真实页数
    max_score_idx = score.argsort()[::-1] + 1
    questions_top10[question_idx]["reference"] = [
        "page_" + str(x) for x in max_score_idx[:10]
    ]
with open("./Week06/tfidf_retrival_top10.json", "w", encoding="utf-8") as f:
    json.dump(questions_top10, f, ensure_ascii=False, indent=4)
```

RAG_02_RAG检索BM25.py

TF-IDF：明确构造稀疏向量 + L2 归一化 → 点积 = 余弦相似度

BM25：直接用 BM25 非线性公式按词项累加相关性 → 得到检索分数（不是余弦）

In [59]:
```
import json

import jieba
import numpy as np
import pdfplumber
from rank_bm25 import BM25Okapi
```

In [60]:
```
questions = json.load(open("./Week06/questions.json", "r", encoding="utf-8"))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")
pdf_content = []
for page_idx in range(len(pdf.pages)):
    pdf_content.append(
        {
            "page": "page_" + str(page_idx + 1),
            "content": pdf.pages[page_idx].extract_text(),
        }
    )
```

In [66]:
```
# 这里与tfidf不一样，这里每一行是一个token列表，因为其只负责打分，
# 而tfidf每一行是一个“空格分隔token”的字符串，其内部会有tokenizer进行分词，因为to
# 还有一点不一样是tfidf实现里使用问题去建立词表，标准的做法是用文档库建立词典，然
pdf_words = [jieba.lcut(x["content"]) for x in pdf_content]
bm25 = BM25Okapi(pdf_words)
question_top10 = questions

for query_idx, question in enumerate(questions):
    doc_scores = bm25.get_scores(jieba.lcut(question["question"]))
    max_score_page = np.argmax(doc_scores)
    questions[query_idx]["reference"] = "page_" + str(max_score_page)
with open("./Week06/bm25_retrieval_top1.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

In [67]:
```
for query_idx in range(len(questions)):
    doc_scores = bm25.get_scores(jieba.lcut(questions[query_idx]["question"]))
    max_score_page_idx = doc_scores.argsort()[::-1] + 1
    questions[query_idx]["reference"] = [
        "page_" + str(x) for x in max_score_page_idx[:10]
    ]

with open("./Week06/bm25_retrieval_top10.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

RAG_04_RAG检索BERT.py

分别使用BGE和Jina模型对文本进行embedding

In [12]:
```
import json
import os
import time

import pdfplumber
from sentence_transformers import SentenceTransformer
```

In [18]:
```
questions = json.load(open("./Week06/questions.json", "r", encoding="utf-8"))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")
pdf_content = []
```

```
for page_idx in range(len(pdf.pages)):
    pdf_content.append(
        {
            "page": "page_" + str(page_idx + 1),
            "content": pdf.pages[page_idx].extract_text(),
        }
    )
```

In [14]:
```
pdf_content = [x["content"] for x in pdf_content]
questions_content = [x["question"] for x in questions]

# 使用BGE进行embedding
start_time = time.time()
bert_base_chinese = os.path.join(
    os.getenv("MODEL_PATH"), "google-bert", "bert-base-chinese"
)
model = SentenceTransformer(bert_base_chinese)
pdf_embeddings = model.encode(pdf_content, normalize_embeddings=True)
question_embeddings = model.encode(questions_content, normalize_embeddings=True)
end_time = time.time()
elapsed_time = end_time - start_time
print(f"BGE embedding 时间:{elapsed_time:.4f} 秒")
```

```
No sentence-transformers model found with name D:\桌面\typora文件\八斗AI\models\g
oogle-bert\bert-base-chinese. Creating a new one with mean pooling.
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
BertModel LOAD REPORT from: D:\桌面\typora文件\八斗AI\models\google-bert\bert-bas
e-chinese
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.dense.weight       | UNEXPECTED |  |
cls.predictions.transform.dense.bias         | UNEXPECTED |  |
cls.seq_relationship.weight                  | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |  |
cls.predictions.bias                         | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; no
t ok if you expect identical arch.
BGE embedding 时间:66.8695 秒
```

In [15]:
```
start_time1 = time.time()
for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1][0] + 1
    questions[query_idx]["reference"] = "page_" + str(max_score_page_idx)

with open("./Week06/bge_retrieval_top1.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
end_time1 = time.time()
elapsed_time1 = end_time1 - start_time1
print(f"BGE top1 检索时间:{elapsed_time1:.4f} 秒")

start_time2 = time.time()
for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1] + 1
```

```
    questions[query_idx]["reference"] = [
        "page_" + str(x) for x in max_score_page_idx[:10]
    ]

with open("./Week06/bge_retrieval_top10.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
end_time2 = time.time()
elapsed2_time = end_time2 - start_time2
print(f"BGE top10 检索时间:{elapsed2_time:.4f} 秒")
```

```
BGE top1 检索时间:0.0458 秒
BGE top10 检索时间:0.0181 秒
```

In [7]:

```
# 使用Jina进行ebedding
```

In [19]:

```
start_time = time.time()
Jina = os.path.join(os.getenv("MODEL_PATH"), "jina", "jina-embeddings-v2-base-zh
model = SentenceTransformer(Jina)
question_sentences = [x["question"] for x in questions]
pdf_content_sentences = [x["content"] for x in pdf_content]

question_embeddings = model.encode(question_sentences, normalize_embeddings=True
pdf_embeddings = model.encode(pdf_content_sentences, normalize_embeddings=True)
end_time = time.time()
elapsed_time = end_time - start_time
print(f"Jina embedding 时间:{elapsed_time:.4f} 秒")

start_time1 = time.time()
for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1][0] + 1
    questions[query_idx]["reference"] = "page_" + str(max_score_page_idx)

with open("submit_jina_retrieval_top1.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
end_time1 = time.time()
elapsed_time1 = end_time1 - start_time1
print(f"Jina top1 检索时间:{elapsed_time1:.4f} 秒")

start_time2 = time.time()
for query_idx, feat in enumerate(question_embeddings):
    score = feat @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1] + 1
    questions[query_idx]["reference"] = [
        "page_" + str(x) for x in max_score_page_idx[:10]
    ]

with open("submit_jina_retrieval_top10.json", "w", encoding="utf8") as up:
    json.dump(questions, up, ensure_ascii=False, indent=4)
end_time2 = time.time()
elapsed2_time = end_time2 - start_time2
print(f"Jina top10 检索时间:{elapsed2_time:.4f} 秒")
```

```
Loading weights:   0%|          | 0/126 [00:00<?, ?it/s]
```

```
BertModel LOAD REPORT from: D:\桌面\typora文件\八斗AI\models\jina\jina-embeddings
-v2-base-zh
Key                                                | Status     |
---------------------------------------------------+------------+-
encoder.layer.{0...11}.mlp.layernorm.bias          | UNEXPECTED |
encoder.layer.{0...11}.mlp.wo.bias                 | UNEXPECTED |
encoder.layer.{0...11}.mlp.layernorm.weight        | UNEXPECTED |
encoder.layer.{0...11}.mlp.gated_layers.weight     | UNEXPECTED |
encoder.layer.{0...11}.mlp.wo.weight               | UNEXPECTED |
encoder.layer.{0...11}.output.dense.bias           | MISSING    |
encoder.layer.{0...11}.output.dense.weight         | MISSING    |
encoder.layer.{0...11}.output.LayerNorm.weight     | MISSING    |
encoder.layer.{0...11}.intermediate.dense.bias     | MISSING    |
encoder.layer.{0...11}.output.LayerNorm.bias       | MISSING    |
encoder.layer.{0...11}.intermediate.dense.weight   | MISSING    |
embeddings.position_embeddings.weight              | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; no
t ok if you expect identical arch.
- MISSING       :those params were newly initialized because missing from the che
ckpoint. Consider training on your downstream task.
Jina embedding 时间:767.3063 秒
Jina top1 检索时间:0.7358 秒
Jina top10 检索时间:0.8387 秒
```

BGE 和 Jina 模型对比

9. 详细对比：BGE（bge-small-zh-v1.5） vs Jina（jina-embeddings-v2-base-zh）

下面的比较以你脚本里用到的两款模型为准：

- `BAAI/bge-small-zh-v1.5`
- `jinaai/jina-embeddings-v2-base-zh`

9.1 定位与适用语言

BGE（bge-small-zh-v1.5）

- **中文为主**的通用向量模型系列（BGE = BAAI General Embedding）。
- 更典型的用法是：中文 query 检索中文语料；也可用于一定程度的中英，但它不是专门的双语对齐模型。

Jina v2 base zh

- **中英双语（bilingual）**向量空间设计：同义的中英文句子倾向映射到同一 embedding 空间。
- 典型用法：
  - 中文搜中文 / 英文搜英文（单语）
  - 中文搜英文、英文搜中文（跨语）

9.2 维度、上下文长度、模型规模（这些会直接影响速度与“长文档处理方式”）

BGE small zh v1.5

- **Embedding 维度：512**
- **最大序列长度：通常按 512 token 截断**（很多示例/评测也是 512）
- 模型体量相对更小 → 推理更快、更省显存

Jina embeddings v2 base zh

- **Embedding 维度：768**
- **最大序列长度：8192**（长上下文是它的核心卖点之一）
- **参数量：161M**（base 级别）

> 直观结论：
>
> - 你的文档如果切得很短（几百字一个 chunk），BGE small 往往性价比很好。
> - 如果你希望"更长 chunk / 更少切分"或需要跨中英检索，Jina 的优势会更明显。

9.3 pooling/向量提取方式（两者一个常见坑）

BGE：更常见的是 CLS pooling

- 在不少 BGE 的 transformer 用法示例中，会直接取 `model_output[0][:,0]`（CLS token）作为句向量，然后再 L2 normalize。
- SentenceTransformer 封装时也会按它的配置做 pooling（你在脚本里只要 `encode()` 就行）。

Jina：官方强调 **mean pooling**

- Jina 的模型卡明确建议：做 sentence embedding 时使用 mean pooling（对所有 token embedding 按 attention mask 做平均），再 L2 normalize。
- 这意味着：如果你不走它提供的 `encode()`（或不按官方 pooling），效果可能会明显下降。

> 你现在这份脚本是用 SentenceTransformer 的 `encode()`，通常会帮你处理 pooling；但当你换成 `transformers.AutoModel` 自己写推理时，这一点就非常关键。

9.4 是否需要"检索指令（instruction / prompt）"

BGE：有经典的"query instruction"模式

- BGE 系列常见建议：
  - **短 query → 长 passage** 的检索任务，可以给 query 加一个前缀指令（中文模型常见类似"为这个句子生成表示以用于检索相关文章："）。
  - passage/文档一般不加指令。
- v1.5 版本强调：**不加指令也能用**，只是某些任务上加指令会更强。

Jina：更强调"天然双语/长上下文"，不突出指令范式

- Jina v2 的模型卡更多强调跨语、长上下文以及 pooling 方式。
- 你一般直接丢原始 query / doc 就可以开始用。

9.5 相似度分数的“可解释性”与分布

两者在你脚本里都用了 `normalize_embeddings=True`，因此：

- 最终排序分数本质上都是 cosine（用点积实现）

但注意：

- 不同模型的相似度分布可能很不一样（例如 BGE 官方就提到过相似度分布偏高的问题，并在 v1.5 做了改善）。
- 所以在检索里：**相似度绝对值不要迷信**，最重要的是“排序相对关系”。

9.6 在你的“按页检索”任务里怎么选（可操作建议）

什么时候更偏向选 BGE small zh v1.5

- 语料是中文为主
- 你会把 PDF 切成较短 chunk（例如 200~800 字）
- 更在意速度/成本（小模型 + 512 维）

什么时候更偏向选 Jina v2 base zh

- 文档 chunk 想切得更长（或者你希望减少切分数量）
- 有明显中英混合内容，或需要跨语检索
- 接受更高的推理成本（768 维、base 模型、长上下文）

9.7 一个“更强”的实践：Hybrid + Rerank

不管你选 BGE 还是 Jina，在 RAG 里常见最强组合是：

1. 召回：BM25（词面精确） + Embedding（语义召回）融合
2. 重排：cross-encoder reranker（例如 bge-reranker）对 TopK 做精排

这样通常能同时解决：

- 术语/零件号/型号（BM25 强）
- 语义改写/同义表达（embedding 强）
- 最终排序精度（reranker 强）

RAG_04_RAG检索BERT_Segment.py

相较于前面的代码不是以一页为chunk，而是40字符为一个chunk

In [25]:
```
import json
import os
import pdfplumber
from sentence_transformers import SentenceTransformer

# 读取pdf和json文件
questions = json.load(open("./Week06/questions.json", "r", encoding="utf-8"))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")

# pdf chunk 划分
def split_text_fixed_size(text: str, chunk_size=40):
```

```
    return [text[i : i + chunk_size] for i in range(0, len(text), chunk_size)]

# pdf chunk切分，每条记录长这样
{'page': 'page_17', 'content': '这页里的第 3 个 chunk 的 40 字…'}
pdf_content = []
for page_idx in range(len(pdf.pages)):
    for chunk in split_text_fixed_size(pdf.pages[page_idx].extract_text()):
        pdf_content.append({
            'page':'page_'+str(page_idx),
            'content':chunk
        })
```

In [38]:

```
questions_text = [x['question'] for x in questions]
pdf_text = [x['content'] for x in pdf_content]

bert_path = os.path.join(os.getenv('MODEL_PATH'),'sentence_bert','bge-small-zh-v
model = SentenceTransformer(bert_path)
questions_embeddings = model.encode(questions_text,normalize_embeddings=True,sho
pdf_embeddings = model.encode(pdf_text,normalize_embeddings=True,show_progress_b
```

```
Loading weights:   0%|          | 0/71 [00:00<?, ?it/s]
BertModel LOAD REPORT from: D:\桌面\typora文件\八斗AI\models\sentence_bert\bge-sm
all-zh-v1.5
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; no
t ok if you expect identical arch.
Batches:   0%|          | 0/10 [00:00<?, ?it/s]
Batches:   0%|          | 0/117 [00:00<?, ?it/s]
```

In [39]:

```
# top1检索
for query_idx,question in enumerate(questions_embeddings):
    score = question @ pdf_embeddings.T
    max_score_page = score.argsort()[-1]
    questions[query_idx]['reference'] = pdf_content[max_score_page]['page']
# 以前：max_score_page_idx + 1 是页码
# 现在：max_score_page_idx 是 chunk 的索引
# 最终页码来自：pdf_content[chunk_idx]['page']
with open('./Week06/bge_sgement_retrieval_top1.json', 'w', encoding='utf8') as u
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

In [43]:

```
# top10检索
# 对重复的页面去重
def remove_duplicates(input_list):
    seen = set()
    result = []
    for item in input_list:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
for query_idx,question in enumerate(questions_embeddings):
    score = question @ pdf_embeddings.T
    max_score_page_idx = score.argsort()[::-1]
    pages = [pdf_content[x]['page'] for x in max_score_page_idx]
    # 取前100个chunk对应的页面，最后得到top10
```

```
    questions[query_idx]['reference'] = remove_duplicates(pages[:100])[:10]
with open('./Week06/bge_sgement_retrieval_top10.json', 'w', encoding='utf8') as
    json.dump(questions, up, ensure_ascii=False, indent=4)
```

RAG_06_RAG重排序.py

In [46]:

```
import json
import pdfplumber
from transformers import AutoModelForSequenceClassification, AutoTokenizer
import torch

# 读取数据集
questions = json.load(open("./Week06/questions.json","r",encoding='utf-8'))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")
pdf_content = []
for page_idx in range(len(pdf.pages)):
    pdf_content.append({
        'page': 'page_' + str(page_idx + 1),
        'content': pdf.pages[page_idx].extract_text()
    })

# 加载排序模型
bert_path = os.path.join(os.getenv('MODEL_PATH'),'sentence_bert','bge-small-zh-v
tokenizer = AutoTokenizer.from_pretrained(bert_path)
rerank_model = AutoModelForSequenceClassification.from_pretrained(bert_path)
# rerank_model.cuda()
rerank_model.eval()

# 进行召回合并
bge = json.load(open('./Week06/bge_sgement_retrieval_top10.json','r',encoding='u
bm25 = json.load(open('./Week06/bm25_retrieval_top10.json','r',encoding='utf-8')

fusion_result = []
k = 60 # 超参数，实现发现60
for q1, q2 in zip(bge, bm25):
    # q1 每个提问 的 bge 检索的top10
    # q2 每个提问 的 bm25 检索的top10
    print(len(fusion_result), len(bge))

    # 多路检索的 结果的合并
    fusion_score = {} # 每个页面 最终的打分
    for idx, q in enumerate(q1['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k) # 排在后面，得分更低
        else:
            fusion_score[q] += 1 / (idx + k)

    for idx, q in enumerate(q2['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    sorted_dict = sorted(fusion_score.items(), key=lambda item: item[1], reverse

    pairs = []
    for sorted_result in sorted_dict[:3]:
        page_index = int(sorted_result[0].split('_')[1]) - 1
```

```
        # 新的样本：【提问， 检索到的文档页面】
        pairs.append([q1["question"], pdf_content[page_index]['content']])

    inputs = tokenizer(pairs, padding=True, truncation=True, return_tensors='pt'
    with torch.no_grad():
        inputs = {key: inputs[key] for key in inputs.keys()}
        scores = rerank_model(**inputs, return_dict=True).logits.view(-1, ).floa

    # .cpu 数据 从gpu 移动到cpu进行计算
    sorted_result = sorted_dict[scores.cpu().numpy().argmax()]
    q1['reference'] = sorted_result[0]

    fusion_result.append(q1)

with open('./Week06/fusion_bge+bm25_rerank_retrieval.json', 'w', encoding='utf8'
    json.dump(fusion_result, up, ensure_ascii=False, indent=4)
```

```
Loading weights:   0%|          | 0/71 [00:00<?, ?it/s]
BertForSequenceClassification LOAD REPORT from: D:\桌面\typora文件\八斗AI\models
\sentence_bert\bge-small-zh-v1.5
Key                      | Status     |
-------------------------+------------+-
embeddings.position_ids  | UNEXPECTED |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; no
t ok if you expect identical arch.
- MISSING       :those params were newly initialized because missing from the che
ckpoint. Consider training on your downstream task.
```

0 301
1 301
2 301
3 301
4 301
5 301
6 301
7 301
8 301
9 301
10 301
11 301
12 301
13 301
14 301
15 301
16 301
17 301
18 301
19 301
20 301
21 301
22 301
23 301
24 301
25 301
26 301
27 301
28 301
29 301
30 301
31 301
32 301
33 301
34 301
35 301
36 301
37 301
38 301
39 301
40 301
41 301
42 301
43 301
44 301
45 301
46 301
47 301
48 301
49 301
50 301
51 301
52 301
53 301
54 301
55 301
56 301
57 301
58 301
59 301

60 301
61 301
62 301
63 301
64 301
65 301
66 301
67 301
68 301
69 301
70 301
71 301
72 301
73 301
74 301
75 301
76 301
77 301
78 301
79 301
80 301
81 301
82 301
83 301
84 301
85 301
86 301
87 301
88 301
89 301
90 301
91 301
92 301
93 301
94 301
95 301
96 301
97 301
98 301
99 301
100 301
101 301
102 301
103 301
104 301
105 301
106 301
107 301
108 301
109 301
110 301
111 301
112 301
113 301
114 301
115 301
116 301
117 301
118 301
119 301

120 301
121 301
122 301
123 301
124 301
125 301
126 301
127 301
128 301
129 301
130 301
131 301
132 301
133 301
134 301
135 301
136 301
137 301
138 301
139 301
140 301
141 301
142 301
143 301
144 301
145 301
146 301
147 301
148 301
149 301
150 301
151 301
152 301
153 301
154 301
155 301
156 301
157 301
158 301
159 301
160 301
161 301
162 301
163 301
164 301
165 301
166 301
167 301
168 301
169 301
170 301
171 301
172 301
173 301
174 301
175 301
176 301
177 301
178 301
179 301

180 301
181 301
182 301
183 301
184 301
185 301
186 301
187 301
188 301
189 301
190 301
191 301
192 301
193 301
194 301
195 301
196 301
197 301
198 301
199 301
200 301
201 301
202 301
203 301
204 301
205 301
206 301
207 301
208 301
209 301
210 301
211 301
212 301
213 301
214 301
215 301
216 301
217 301
218 301
219 301
220 301
221 301
222 301
223 301
224 301
225 301
226 301
227 301
228 301
229 301
230 301
231 301
232 301
233 301
234 301
235 301
236 301
237 301
238 301
239 301

240 301
241 301
242 301
243 301
244 301
245 301
246 301
247 301
248 301
249 301
250 301
251 301
252 301
253 301
254 301
255 301
256 301
257 301
258 301
259 301
260 301
261 301
262 301
263 301
264 301
265 301
266 301
267 301
268 301
269 301
270 301
271 301
272 301
273 301
274 301
275 301
276 301
277 301
278 301
279 301
280 301
281 301
282 301
283 301
284 301
285 301
286 301
287 301
288 301
289 301
290 301
291 301
292 301
293 301
294 301
295 301
296 301
297 301
298 301

```
299 301
300 301
```

07_RAG多路召回

In [47]:
```
import json
bge = json.load(open('./Week06/bge_sgement_retrieval_top10.json','r',encoding='u
bm25 = json.load(open('./Week06/bm25_retrieval_top10.json','r',encoding='utf-8')

fusion_result = []
k = 60
for q1, q2 in zip(bge, bm25):
    fusion_score = {}
    for idx, q in enumerate(q1['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    for idx, q in enumerate(q2['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    sorted_dict = sorted(fusion_score.items(), key=lambda item: item[1], reverse
    q1['reference'] = sorted_dict[0][0]
    fusion_result.append(q1)

with open('./Week06/fusion_bge+bm25_retrieval.json', 'w', encoding='utf8') as up
    json.dump(fusion_result, up, ensure_ascii=False, indent=4)
```

08_RAG问答

In [ ]:
```
import json
import pdfplumber
import time
import jwt
import requests
import numpy as np
import os

# 实际KEY，过期时间
def generate_token(apikey: str, exp_seconds: int):
    try:
        id, secret = apikey.split(".")
    except Exception as e:
        raise Exception("invalid apikey", e)

    payload = {
        "api_key": id,
        "exp": int(round(time.time() * 1000)) + exp_seconds * 1000,
        "timestamp": int(round(time.time() * 1000)),
    }
    return jwt.encode(
        payload,
        secret,
```

```
        algorithm="HS256",
        headers={"alg": "HS256", "sign_type": "SIGN"},
    )

def ask_glm(content):
    url = "https://open.bigmodel.cn/api/paas/v4/chat/completions"
    headers = {
      'Content-Type': 'application/json',
      'Authorization': generate_token("89e209c17a582a029626f784c1fd0777.gk44xUlg
    }

    data = {
        "model": "glm-3-turbo",
        "messages": [{"role": "user", "content": content}]
    }

    response = requests.post(url, headers=headers, json=data)
    print("【大模型返回】")
    print(response)
    return response.json()

def ask_gpt(content):
    url = "https://openai.api2d.net/v1/chat/completions"
    headers = {
      'Content-Type': 'application/json',
      'Authorization': "Bearer fk222903-VSeeeE4nL4I7MUbcuJEGpUEkVH6VgEnP"
    }

    max_error = 0
    while True:
        if max_error > 5:
            return "无法回答"
        try:
            data = {
                "model": "gpt-3.5-turbo-0613",
                "messages": [{"role": "user", "content": content}]
            }

            response = requests.post(url, headers=headers, json=data, timeout=10
            # print("【大模型返回】\n" + response)
            return response.json()
        except:
            max_error += 1
            continue
def ask_deepseek(content):
    api_key = os.getenv("DEEPSEEK_API_KEY")
    url = "https://api.deepseek.com/chat/completions"
    headers = {
      'Content-Type': 'application/json',
      'Authorization': "Bearer "+api_key
    }

    max_error = 0
    while True:
        if max_error > 5:
            return "无法回答"
        try:
            data = {
                "model": "deepseek-chat",
                "messages": [{"role": "user", "content": content}]
```

```
            }

            response = requests.post(url, headers=headers, json=data, timeout=10
            # print("【大模型返回】\n" + response)
            return response.json()
        except:
            max_error += 1
            continue

bge = json.load(open('./Week06/bge_sgement_retrieval_top10.json','r',encoding='u
bm25 = json.load(open('./Week06/bm25_retrieval_top10.json','r',encoding= 'utf-8'

questions = json.load(open("./Week06/questions.json",'r',encoding='utf-8'))
pdf = pdfplumber.open("./Week06/汽车知识手册.pdf")
pdf_content_dict = {}
for page_idx in range(len(pdf.pages)):
    pdf_content_dict['page_' + str(page_idx + 1)] = pdf.pages[page_idx].extract_

fusion_result = []
k = 60
for q1, q2 in zip(bge[:], bm25[:]):
    fusion_score = {}
    for idx, q in enumerate(q1['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    for idx, q in enumerate(q2['reference']):
        if q not in fusion_score:
            fusion_score[q] = 1 / (idx + k)
        else:
            fusion_score[q] += 1 / (idx + k)

    sorted_dict = sorted(fusion_score.items(), key=lambda item: item[1], reverse
    q1['reference'] = sorted_dict[0][0]

    reference_pages = [int(x[0].split('_')[1]) for x in sorted_dict]
    reference_pages = np.array(reference_pages)

    reference_content = ''
    reference_content += pdf_content_dict[sorted_dict[0][0]].replace('\n', ' ')

    print("【用户提问】\n" + q1["question"])
    print("【参考资料】\n" + pdf_content_dict[q1['reference']])

    reference_page = q1['reference'].split('_')[1]

    prompt = '''你是一个汽车专家，你擅长编写和回答汽车相关的用户提问，帮我结合给定
如果问题无法从资料中获得，或无法从资料中进行回答，请回答无法回答。如果提问不符合逻
如果问题可以从资料中获得，则请逐步回答。

资料：{0}


问题：{1}
    '''.format(reference_content, q1["question"])
    # answer = ask_glm(prompt)['choices'][0]['message']['content']

    answer = '无法'
```

```
    for _ in range(5):
        try:
            answer = ask_deepseek(prompt)['choices'][0]['message']['content']
            if answer:
                break
        except:
            continue

    if '无法' in answer:
        answer = '结合给定的资料，无法回答问题。'

    q1['answer'] = answer
    print("【模型回答】\n" + q1['answer'])

    print("\n\n\n")

    fusion_result.append(q1)

with open('./Week06/fusion_bge+bm25_rerank_retrieval_glm4.json', 'w', encoding='
    json.dump(fusion_result, up, ensure_ascii=False, indent=4)
```

【用户提问】
“前排座椅通风”的相关内容在第几页？
【参考资料】
安全出行
在中央显示屏中点击 ，进入驾驶员侧座椅通风控制界面。
01设置副驾驶员侧座椅通风强度及开关控制。
驾驶员/副驾驶员侧座椅通风分三级调节，点击控制开关后在“关-低-
01设置驾驶员侧座椅通风强度及开关控制。
中-高”之间循环。
在中央显示屏中点击 ，进入副驾驶员侧座椅通风控制界面。
使用Lynk&CoApp打开/关闭前排座椅通风
打开/关闭前排座椅通风图标：登录Lynk&CoApp，按下
该图标可以打开/关闭前排座椅通风。
设置前排座椅通风时间
在中央显示屏中唤起空调控制界面，然后点击 -舒适。
116
【模型回答】
根据您提供的资料，“前排座椅通风”的相关内容出现在第116页。

【用户提问】
"关于车辆的儿童安全座椅固定装置，在哪一页可以找到相关内容？"
【参考资料】
安全出行
儿童座椅固定装置
儿童安全
儿童乘车时，请务必时刻确保儿童乘坐安全。
婴儿和儿童必须乘坐在适合其年龄和体型的儿童安全座椅中。
01儿童锁已停用，可以从车内和车外打开后门。
02儿童锁已启用，不能从车内打开后门。
儿童锁位于后门的后缘，后门打开时方可触及儿童锁。
可单独启用后排儿童锁：
使用可分开的机械钥匙，顺时针转动控制器90°（左后门）。 儿童座椅警告标贴
不得在受正面安全气囊保护（激活状态下）的座位上使用后向儿童约
束系统。
说明！
□ 对于右后门的儿童锁，逆时针转动钥匙90°，启用儿童锁。
按相反方向转动相应控制器，可以停用儿童锁。
122
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
“打开前机舱盖”的相关信息在第几页？
【参考资料】
保养和维护
驾驶员座椅下方的保险丝盒
乘客舱内的保险丝盒（CJB）位于驾驶员座椅下方，可从后排进行保
险丝的相关操作。
1 将驾驶员座椅尽量前移。
2释放地毯锁片，打开护罩。
3松开保险丝盒盖卡扣。
说明！
□ 所有保险丝的操作应该由Lynk&Co领克中心的专业人员进行。
OBD-II插座位于仪表板下方

使用诊断工具读取VIN码 2连接车辆诊断系统。
3打开点火开关。
1 连接车载诊断（OBD-II）插座。
4诊断系统自动读取VIN码。
打开前机舱盖
打开前机舱盖
打开前机舱盖前，确保前机舱盖打开区域无障碍物。
1 连续拉两次位于驾驶员侧仪表板左下方的前机舱盖开启拉手。
307
【模型回答】
根据您提供的资料，“打开前机舱盖”的相关信息位于第307页。

【用户提问】
“打开前机舱盖”这个操作在哪一页？
【参考资料】
保养和维护
驾驶员座椅下方的保险丝盒
乘客舱内的保险丝盒（CJB）位于驾驶员座椅下方，可从后排进行保
险丝的相关操作。
1 将驾驶员座椅尽量前移。
2释放地毯锁片，打开护罩。
3松开保险丝盒盖卡扣。
说明！
□ 所有保险丝的操作应该由Lynk&Co领克中心的专业人员进行。
OBD-II插座位于仪表板下方
使用诊断工具读取VIN码 2连接车辆诊断系统。
3打开点火开关。
1 连接车载诊断（OBD-II）插座。
4诊断系统自动读取VIN码。
打开前机舱盖
打开前机舱盖
打开前机舱盖前，确保前机舱盖打开区域无障碍物。
1 连续拉两次位于驾驶员侧仪表板左下方的前机舱盖开启拉手。
307
【模型回答】
根据您提供的资料，“打开前机舱盖”这个操作在**第307页**。

【用户提问】
“查看行车记录仪视频”这一项内容在第几页？
【参考资料】
中央显示屏
操作行车记录仪 说明！
行车记录仪为您提供个性化体验以及多种功能。例如：紧急录像、拍
照等。 □ 行车记录仪开启后，行车记录仪 图标会点亮红点。
□ 如遇碰撞等紧急情况时，行车记录仪将进入紧急录像模式，中央
显示屏会发出提示音，并保存紧急视频触发前15秒和触发后15
手动开启/关闭循环录像功能
秒的视频，当紧急视频录制完成后，行车记录仪重新回到触发前
的状态。
在中央显示屏中点击 -行车记录仪，进入行车记录仪设置界面。
□ 当点火模式处于I或车辆处于启动状态时，行车记录仪会自动启
用循环录像功能。
□ 方向盘自定义按键若设置为行车记录仪时，短按为拍照，长按为
紧急录像。

设置行车记录仪
在中央显示屏中先点击 -行车记录仪，再点击 图标，进入行车记
录仪设置界面。
01点击开启/关闭音频录制。
02点击开启/关闭行车记录仪循环录像功能。
274
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
请问Lynk&Co领克汽车的事件数据记录系统（EDR）主要记录哪些信息？
【参考资料】
前言
本手册相关的重要信息 为了凸显重点、便于阅读，中央显示屏中的插图突出显示重点部位，
领克汽车销售有限公司（下称“Lynk&Co领克”）建议您在首次使用 其他部位做了一些处理，提
示您不必关注这些部位。
车辆前，认真阅读本手册内容。为了更好地理解本手册中的内容，您
需要了解以下所有信息。 此图标表示“请勿这样做”或“请勿让此种情况发生”。
提示信息
警告！
人身伤害 此图标表示“建议这样做”或“可以让此种情况发生”。
■ 警告标识提醒您：如未按照该警告内容操作可能会对您或他人造
成人身伤害或生命危险。
注意！ 如果一幅插图显示多个零件，此图标表示零件位置。
车辆损坏风险
■ 注意标识提醒您：如未按照该注意事项操作可能会导致车辆损 如果一幅插图包含两个以上
动作，此图标表示包含顺序的移动/
坏。 动作。
如果一幅插图只包含一个动作，此图标表示移动/动作。
说明！
帮助信息
显示文本
□ 在这里您可以找到一些帮助提示或有用的详细信息。
车辆显示屏上显示的文本或消息的格式与普通文本不同（例如：取
消、系统重置、路径编辑等），程序和指示分多个步骤显示（例如：
车辆-车辆设置）。
图标和插图说明
敬告用户
本手册中使用的插图和视频剪辑仅是示意图，展示车辆的某些特征或
功能，用于辅助说明，以帮助您理解。此外车辆功能也会随着软件版
为了您的安全，请您在驾驶车辆时遵守当地的法律法规，并遵循以下
本升级或功能开通情况进行更迭，相应的内容及插图可能会和实车存
注意事项：
在差异，仅供参考，具体情况请以实车为准。
11
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
问题：事件数据记录系统（EDR）中的数据是否可以被黑客利用进行恶意攻击？
【参考资料】
前言
本手册相关的重要信息 为了凸显重点、便于阅读，中央显示屏中的插图突出显示重点部位，
领克汽车销售有限公司（下称“Lynk&Co领克”）建议您在首次使用 其他部位做了一些处理，提

示您不必关注这些部位。
车辆前，认真阅读本手册内容。为了更好地理解本手册中的内容，您
需要了解以下所有信息。 此图标表示“请勿这样做”或“请勿让此种情况发生”。
提示信息
警告！
人身伤害 此图标表示“建议这样做”或“可以让此种情况发生”。
■ 警告标识提醒您：如未按照该警告内容操作可能会对您或他人造
成人身伤害或生命危险。
注意！ 如果一幅插图显示多个零件，此图标表示零件位置。
车辆损坏风险
■ 注意标识提醒您：如未按照该注意事项操作可能会导致车辆损 如果一幅插图包含两个以上
动作，此图标表示包含顺序的移动/
坏。 动作。
如果一幅插图只包含一个动作，此图标表示移动/动作。
说明！
帮助信息
显示文本
□ 在这里您可以找到一些帮助提示或有用的详细信息。
车辆显示屏上显示的文本或消息的格式与普通文本不同（例如：取
消、系统重置、路径编辑等），程序和指示分多个步骤显示（例如：
车辆-车辆设置）。
图标和插图说明
敬告用户
本手册中使用的插图和视频剪辑仅是示意图，展示车辆的某些特征或
功能，用于辅助说明，以帮助您理解。此外车辆功能也会随着软件版
为了您的安全，请您在驾驶车辆时遵守当地的法律法规，并遵循以下
本升级或功能开通情况进行更迭，相应的内容及插图可能会和实车存
注意事项：
在差异，仅供参考，具体情况请以实车为准。
11
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
问题：在国家环保法要求下，哪些情况下需要对车辆进行报废处理？
【参考资料】
前言
原厂精装附件、选装装备和改装 在使用无线电设备时应当严格遵守以下规定：
■ 切勿自行安装或变更无线电设备，否则可能会造成某些电气系统
Lynk&Co领克原厂精装附件都是经过测试的，确保能够与您车辆的性 故障。
能、安全和排放系统很好地兼容。为了尽可能的保证车辆性能与安
■ 使用无线电设备时，请遵循车辆所在地法规。
全，建议您优先使用Lynk&Co领克原厂精装附件。
■ 切勿将车辆停放在变压器等电磁干扰较强的地方。
如果您安装非Lynk&Co领克原厂精装附件，可能会对您车辆的性能和
安全系统产生不利影响。因此产生的任何问题，Lynk&Co领克将不会
警告！
承担责任。
■ 使用植入式心脏起搏器、心脏再同步治疗起搏器或植入式心脏复
您可以订购某些经过Lynk&Co领克认证的选装装备。由于销售市场不 律除颤器的人士应与无线
电设备保持适当的距离。无线电波可能
同，专门为某些国家或地区提供的选装装备可能不适用于您的地区。 影响此类设备的工作。
如有必要，请禁用无线电设备。
在您订购选装装备之前，请查询当地法律法规，并联系Lynk&Co领克 ■ 请勿将无线电设备安装
在安全气囊展开的范围内。
中心。如果您使用了未经Lynk&Co领克认证或不适用您所在地区的选
■ 对于使用其他电子医疗设备的人士，应向该设备制造商咨询无线

装装备，引起的任何问题，Lynk&Co领克将不会承担责任。
电波对设备工作的影响情况。无线电波可能会对这类医疗设备的
工作产生难以预料的影响。
切勿私自改装车辆，否则将会影响车辆的操控性、安全性和耐久性，
同时有可能会触犯当地政府条例。因私自改装引起的车辆损坏或性能
问题，Lynk&Co领克将不承担任何责任。 注意！
■ 自行安装或变更无线电设备所造成的影响和损失，Lynk&Co领克
警告！ 将不承担任何责任。
■ 无线电设备的电波，会对安全气囊的控制装置产生影响。
■ Lynk&Co领克只对经Lynk&Co领克测试且认证的原厂精装附件
和选装装备负责。因安装非原厂精装附件或非选装装备而导致的
人身伤亡、车损或费用，Lynk&Co领克不提供任何保证且不承担
任何责任，不履行任何义务。
所有权变更
如果您需要变更车辆所有权，请携带本人身份证明，前往Lynk&Co领
克中心，将用户数据和系统设置恢复出厂设置，终止紧急救援和智能
互联服务，这一点很重要。
无线电设备
在车辆上安装无线电设备可能会影响电气系统的正常使用。
13
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
请问，如果车辆报废后，原车主是否还能使用该车辆的智能互联服务？
【参考资料】
前言
原厂精装附件、选装装备和改装 在使用无线电设备时应当严格遵守以下规定：
■ 切勿自行安装或变更无线电设备，否则可能会造成某些电气系统
Lynk&Co领克原厂精装附件都是经过测试的，确保能够与您车辆的性 故障。
能、安全和排放系统很好地兼容。为了尽可能的保证车辆性能与安
■ 使用无线电设备时，请遵循车辆所在地法规。
全，建议您优先使用Lynk&Co领克原厂精装附件。
■ 切勿将车辆停放在变压器等电磁干扰较强的地方。
如果您安装非Lynk&Co领克原厂精装附件，可能会对您车辆的性能和
安全系统产生不利影响。因此产生的任何问题，Lynk&Co领克将不会
警告！
承担责任。
■ 使用植入式心脏起搏器、心脏再同步治疗起搏器或植入式心脏复
您可以订购某些经过Lynk&Co领克认证的选装装备。由于销售市场不 律除颤器的人士应与无线
电设备保持适当的距离。无线电波可能
同，专门为某些国家或地区提供的选装装备可能不适用于您的地区。 影响此类设备的工作。
如有必要，请禁用无线电设备。
在您订购选装装备之前，请查询当地法律法规，并联系Lynk&Co领克 ■ 请勿将无线电设备安装
在安全气囊展开的范围内。
中心。如果您使用了未经Lynk&Co领克认证或不适用您所在地区的选
■ 对于使用其他电子医疗设备的人士，应向该设备制造商咨询无线
装装备，引起的任何问题，Lynk&Co领克将不会承担责任。
电波对设备工作的影响情况。无线电波可能会对这类医疗设备的
工作产生难以预料的影响。
切勿私自改装车辆，否则将会影响车辆的操控性、安全性和耐久性，
同时有可能会触犯当地政府条例。因私自改装引起的车辆损坏或性能
问题，Lynk&Co领克将不承担任何责任。 注意！
■ 自行安装或变更无线电设备所造成的影响和损失，Lynk&Co领克
警告！ 将不承担任何责任。
■ 无线电设备的电波，会对安全气囊的控制装置产生影响。
■ Lynk&Co领克只对经Lynk&Co领克测试且认证的原厂精装附件

和选装装备负责。因安装非原厂精装附件或非选装装备而导致的
人身伤亡、车损或费用，Lynk&Co领克不提供任何保证且不承担
任何责任，不履行任何义务。
所有权变更
如果您需要变更车辆所有权，请携带本人身份证明，前往Lynk&Co领
克中心，将用户数据和系统设置恢复出厂设置，终止紧急救援和智能
互联服务，这一点很重要。
无线电设备
在车辆上安装无线电设备可能会影响电气系统的正常使用。
13
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保用车前的准备工作万无一失？
【参考资料】
用车前准备
远程查询车辆状况
行程开始前，您可以通过Lynk&CoApp查看以下车辆状况，以确认是
否满足行程需要：
■ 轮胎胎压
■ 剩余油量
■ 剩余电量
■ 可续航里程
■ 车内温度
■ 车辆位置
安全检查
每次驾车前，请务必注意下列事项：
■ 确保所有车灯和转向指示灯工作正常。
■ 确保所有轮胎压力处于正常范围内。
■ 确保所有车窗玻璃干净、清晰度高。
■ 确保驾驶员座椅、方向盘和后视镜已调节至正确位置。
■ 确保仪表无警告灯点亮或故障警告信息。
■ 确保无物品妨碍踏板行程。
■ 确保所有装运的物件已固定到位。
17
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
装载货物时，车内有哪些储物空间可以利用？
【参考资料】
装载货物
第二排储物空间 使用后排中央扶手和杯托
■ 前排座椅靠背储物袋。 将扶手向前拉动并向下打开以使用。
■ 车门储物空间。
■ 后扶手的杯托。 警告！
■ 衣帽钩。
■ 将松散物品装在密闭的储物空间内，确保物品被固定到位。
■ 衣帽钩只能用于悬挂轻量的衣物，不能用于悬挂雨伞等坚硬物
品。
注意！
■ 切勿强行将尺寸不合适的容器卡入杯托。
26

【模型回答】
根据资料，装载货物时车内可用的储物空间包括：

1. **第二排储物空间**
   - 使用后排中央扶手和杯托（将扶手向前拉动并向下打开以使用）。
   - 前排座椅靠背储物袋。
   - 车门储物空间。
   - 后扶手的杯托。
   - 衣帽钩（仅限悬挂轻量衣物，不可挂坚硬物品）。

2. **注意事项**
   - 松散物品应装在密闭的储物空间内并固定到位。
   - 杯托不可强行放入尺寸不合适的容器。

以上信息来源于资料第26页。

【用户提问】
请问车辆手套箱的容量大小是多少？
【参考资料】
装载货物
手套箱打开/关闭 前排中央扶手打开/关闭
打开：按下解锁按钮，手套箱自动打开。手套箱灯点亮。 打开：按照指示拉起扶手。
关闭：推动手套箱中部直到卡合到位，手套箱灯随之熄灭。 关闭：按下扶手，直至其接合到位。
警告！ 注意！
■ 车辆行驶过程中，手套箱应处于关闭状态。 ■ 当扶手关闭时，您可以将其前后调整到合适的位置。
23
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何合理安排前排的储物空间以提高实用性？
【参考资料】
装载货物
第二排储物空间 使用后排中央扶手和杯托
■ 前排座椅靠背储物袋。 将扶手向前拉动并向下打开以使用。
■ 车门储物空间。
■ 后扶手的杯托。 警告！
■ 衣帽钩。
■ 将松散物品装在密闭的储物空间内，确保物品被固定到位。
■ 衣帽钩只能用于悬挂轻量的衣物，不能用于悬挂雨伞等坚硬物
品。
注意！
■ 切勿强行将尺寸不合适的容器卡入杯托。
26
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
请问车辆手套箱的容量大小是多少？

【参考资料】
装载货物
手套箱打开/关闭 前排中央扶手打开/关闭
打开：按下解锁按钮，手套箱自动打开。手套箱灯点亮。 打开：按照指示拉起扶手。
关闭：推动手套箱中部直到卡合到位，手套箱灯随之熄灭。 关闭：按下扶手，直至其接合到位。
警告！ 注意！
■ 车辆行驶过程中，手套箱应处于关闭状态。 ■ 当扶手关闭时，您可以将其前后调整到合适的位置。
23
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何正确使用前排座椅靠背储物袋？
【参考资料】
装载货物
第二排储物空间 使用后排中央扶手和杯托
■ 前排座椅靠背储物袋。 将扶手向前拉动并向下打开以使用。
■ 车门储物空间。
■ 后扶手的杯托。 警告！
■ 衣帽钩。
■ 将松散物品装在密闭的储物空间内，确保物品被固定到位。
■ 衣帽钩只能用于悬挂轻量的衣物，不能用于悬挂雨伞等坚硬物
品。
注意！
■ 切勿强行将尺寸不合适的容器卡入杯托。
26
【模型回答】
根据资料，正确使用前排座椅靠背储物袋的方法如下：

1. **确认储物袋位置**：前排座椅靠背储物袋位于前排座椅的背面。
2. **使用方式**：直接打开或放入物品即可，资料中未提及特殊操作步骤。
3. **注意事项**：资料中强调，应将松散物品装在密闭的储物空间内，并确保物品被固定到位，以防止车辆行驶中物品移动造成危险。

因此，正确使用方式是直接将物品放入袋中，并确保物品固定稳妥。

【用户提问】
请问，如果后排中央扶手和杯托被占用，驾驶员应该如何放置饮料？
【参考资料】
装载货物
手套箱打开/关闭 前排中央扶手打开/关闭
打开：按下解锁按钮，手套箱自动打开。手套箱灯点亮。 打开：按照指示拉起扶手。
关闭：推动手套箱中部直到卡合到位，手套箱灯随之熄灭。 关闭：按下扶手，直至其接合到位。
警告！ 注意！
■ 车辆行驶过程中，手套箱应处于关闭状态。 ■ 当扶手关闭时，您可以将其前后调整到合适的位置。
25
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何通过手机验证码解锁手套箱？
【参考资料】
装载货物
前排储物空间 副仪表台
■ 遮阳板上的卡片/票据夹。 01无线充电板/置物板
■ 手套箱。
02杯托
■ 车门存储空间。
■ 眼镜盒。 03储物箱
■ 前副仪表台。
■ 前排中央扶手储物箱。 注意！
■ 前副仪表台侧网兜。 ■ 切勿强行将尺寸不合适的容器卡入杯托。
22
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
问题：如果手套箱被锁止了，但是忘记了密码，应该怎么办？
【参考资料】
上车和下车
使用遥控钥匙解锁和闭锁 靠近解锁远离上锁功能
开启靠近解锁远离上锁功能，无需您取出遥控钥匙即可解锁和锁止车
辆。
■ 解锁：车辆处于锁止状态时，携带遥控钥匙靠近车辆时，车辆将
自动解锁。
■ 锁止：所有车门关闭后，携带遥控钥匙远离车辆时，车辆将自动
锁止。
在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
遥控钥匙
您的车辆配备了遥控钥匙。您可以通过遥控钥匙进行以下操作。
锁止按钮：短按锁止按钮，锁止所有车门/尾门，并启用防盗报
警。
解锁按钮：短按解锁按钮，解锁所有车门/尾门，并解除防盗报
警。
尾门按钮：短按尾门按钮，解锁尾门。长按尾门按钮，自动打开/
关闭尾门。尾门自动打开/关闭过程中，短按尾门按钮，尾门将停止 01点击开启/关闭靠近解锁远离上锁功能。
运动。
40
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保尾门在自动打开或关闭过程中安全停止？
【参考资料】
装载货物
仪表板上尾门解锁按钮
01点击打开/关闭尾门。 短按仪表板上的 按钮，解锁尾门。尾门自动打开/关闭过程中，短按该按钮，尾门将停止运动。
长按仪表板上的 按钮，自动打开/关闭尾门。
警告！

■ 尾门自动打开/关闭前，应确保尾门运动范围内无其他人员或障
碍物存在。操作尾门时，切勿以任何形式妨碍尾门动作。
30
【模型回答】
根据您提供的资料，尾门在自动打开或关闭过程中安全停止的方法如下：

逐步回答：

1. **操作前提**：尾门正处于自动打开或关闭的运动过程中。
2. **执行操作**：短按一次仪表板上的尾门解锁按钮（资料中图示为 **01**）。
3. **预期结果**：尾门将立即停止运动。

安全警告（来自资料）：
在操作尾门（包括使其停止）时，务必确保尾门运动范围内没有其他人员或障碍物，并且不要以任何形式妨碍尾门的动作，以确保安全。

答案依据：资料中明确写明："尾门自动打开/关闭过程中，**短按该按钮，尾门将停止运动**。"

【用户提问】
尾门自动打开/关闭的机制是如何感应障碍物的？
【参考资料】
装载货物
仪表板上尾门解锁按钮
01点击打开/关闭尾门。 短按仪表板上的 按钮，解锁尾门。尾门自动打开/关闭过程中，短
按该按钮，尾门将停止运动。
长按仪表板上的 按钮，自动打开/关闭尾门。
警告！
■ 尾门自动打开/关闭前，应确保尾门运动范围内无其他人员或障
碍物存在。操作尾门时，切勿以任何形式妨碍尾门动作。
30
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保尾门在关闭时安全可靠？
【参考资料】
装载货物
使用Lynk&CoApp车外解锁尾门
解锁尾门图标：登录Lynk&CoApp，按下该图标可以解锁
尾门。
走近走远感应式电动尾门
1 在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
01点击按键，开启靠近自动开启功能。
3携带遥控钥匙在距离尾门0.8m左右的区域内站立2秒。
4当转向灯缓慢闪烁时再后退一小步，转向灯再次快速闪烁，尾门
将自动打开。
说明！
□ 如果在转向灯缓慢闪烁时离开车辆感应区域，尾门将不会开启。
□ 当整车处于充电桩、大型停车场、变电站等有信号干扰的地方时
或智能钥匙和干扰设备放在一起时，智能钥匙会受到干扰。因此
01点击按键，开启靠近解锁远离上锁功能。
会偶发出现功能异常情况，当出现异常时，请使用其他方式或者
排除掉干扰后再次尝试。

2在中央显示屏中点击 -车辆设置-电动尾门，进入尾门设置界
□ 为了节约整车功耗，该功能在一些情况下可能会自动关闭，可以
面。
通过启动车辆重新激活。
33
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
尾门在关闭过程中如果遇到障碍物会怎样反应？
【参考资料】
装载货物
使用Lynk&CoApp车外解锁尾门
解锁尾门图标：登录Lynk&CoApp，按下该图标可以解锁
尾门。
走近走远感应式电动尾门
1 在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
01点击按键，开启靠近自动开启功能。
3携带遥控钥匙在距离尾门0.8m左右的区域内站立2秒。
4当转向灯缓慢闪烁时再后退一小步，转向灯再次快速闪烁，尾门
将自动打开。
说明！
□ 如果在转向灯缓慢闪烁时离开车辆感应区域，尾门将不会开启。
□ 当整车处于充电桩、大型停车场、变电站等有信号干扰的地方时
或智能钥匙和干扰设备放在一起时，智能钥匙会受到干扰。因此
01点击按键，开启靠近解锁远离上锁功能。
会偶发出现功能异常情况，当出现异常时，请使用其他方式或者
排除掉干扰后再次尝试。
2在中央显示屏中点击 -车辆设置-电动尾门，进入尾门设置界
□ 为了节约整车功耗，该功能在一些情况下可能会自动关闭，可以
面。
通过启动车辆重新激活。
33
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保电动尾门在运动过程中不会伤及人员或障碍物？
【参考资料】
装载货物
使用Lynk&CoApp车外解锁尾门
解锁尾门图标：登录Lynk&CoApp，按下该图标可以解锁
尾门。
走近走远感应式电动尾门
1 在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
01点击按键，开启靠近自动开启功能。
3携带遥控钥匙在距离尾门0.8m左右的区域内站立2秒。
4当转向灯缓慢闪烁时再后退一小步，转向灯再次快速闪烁，尾门
将自动打开。
说明！
□ 如果在转向灯缓慢闪烁时离开车辆感应区域，尾门将不会开启。
□ 当整车处于充电桩、大型停车场、变电站等有信号干扰的地方时
或智能钥匙和干扰设备放在一起时，智能钥匙会受到干扰。因此
01点击按键，开启靠近解锁远离上锁功能。

会偶发出现功能异常情况，当出现异常时，请使用其他方式或者
排除掉干扰后再次尝试。
2在中央显示屏中点击 -车辆设置-电动尾门，进入尾门设置界
□ 为了节约整车功耗，该功能在一些情况下可能会自动关闭，可以
面。
通过启动车辆重新激活。
33
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
问题：如果尾门在关闭过程中触碰到障碍物，那么尾门会自动关闭到哪个位置？
【参考资料】
装载货物
车外打开/关闭尾门 关闭尾门
打开尾门
尾门关闭按键
侧门关闭（未锁止）时，您可以按下电动尾门下边缘上的 按钮。
尾门开启按键 电动尾门将吸紧关闭。
车辆处于解锁状态时，轻按尾门下部的开启按键，尾门自动打开。
说明！
说明！ □ 尾门开始关闭时，会发出声音信号。
□ 打开/关闭尾门，后备厢灯会相应地自动点亮或熄灭。 □ 尾门打开的状态下锁止车辆，待尾门关闭后，车辆将自动进入锁
止状态。
□ 尾门打开时，车辆将自动点亮后部位置灯，以提醒后方车辆的驾
□ 尾门打开/关闭过程中，按下尾门关闭按键，尾门将停止移动。
驶员，避免发生碰撞。
□ 如果您在短时间内多次打开/关闭尾门，尾门自动打开/关闭功能
会暂时停用，一段时间后才能重新使用。
32
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保在装载货物时汽车的安全性？
【参考资料】
装载货物
注意！ ■ 锁止：点击快捷设置功能界面中的手套箱密码锁按键，在弹出界
面中设定密码，锁止手套箱。
■ 在将座椅靠背向下折叠之前，移除头枕。否则，头枕可能会受到 ■ 解锁：点击快捷设置功能界面中的手套箱密码锁按键或按下手套
损坏。 箱开关，在弹出界面中输入密码解锁手套箱。
■ 折叠座椅靠背前，确保座椅上没有物体且两侧安全带绕开座椅靠
说明！
背，否则座椅可能会受到损坏。
□ 通过密码解锁手套箱后，手套箱密码保护功能自动关闭，如需使
用需重新开启。
使用手套箱密码保护 □ 解锁手套箱时，若密码输入错误超过五次，手套箱密码保护功能
将进入保护状态，2分钟后才可再次输入密码。此时，您也可以
您可以使用密码锁功能锁住手套箱。当您的车辆停在维修站或者使用
通过手机验证码进行解锁。
代客泊车服务时，该功能可以避免手套箱被打开。
启用/停用手套箱密码保护功能

后备厢载物
在中央显示屏中点击 -座舱体验-快捷设置，进入快捷设置界面。
后备厢具备有用于固定行李件的挂钩。
01点击启用/停用手套箱密码保护功能。
28
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
请问这辆正在装载货物的汽车最多可以装载多少立方米的货物？
【参考资料】

【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何设置遥控钥匙的靠近解锁远离上锁功能？
【参考资料】
上车和下车
使用遥控钥匙解锁和闭锁 靠近解锁远离上锁功能
开启靠近解锁远离上锁功能，无需您取出遥控钥匙即可解锁和锁止车
辆。
■ 解锁：车辆处于锁止状态时，携带遥控钥匙靠近车辆时，车辆将
自动解锁。
■ 锁止：所有车门关闭后，携带遥控钥匙远离车辆时，车辆将自动
锁止。
在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
遥控钥匙
您的车辆配备了遥控钥匙。您可以通过遥控钥匙进行以下操作。
锁止按钮：短按锁止按钮，锁止所有车门/尾门，并启用防盗报
警。
解锁按钮：短按解锁按钮，解锁所有车门/尾门，并解除防盗报
警。
尾门按钮：短按尾门按钮，解锁尾门。长按尾门按钮，自动打开/
关闭尾门。尾门自动打开/关闭过程中，短按尾门按钮，尾门将停止 01点击开启/关闭靠近解锁远离上锁功能。
运动。
40
【模型回答】
根据您提供的资料，设置遥控钥匙的“靠近解锁远离上锁”功能，可以按照以下步骤进行：

1. **进入车辆设置界面**：在车辆的**中央显示屏**中，点击进入 **【车辆设置】**。
2. **找到门锁设置**：在车辆设置菜单中，选择进入 **【门锁】** 设置界面。
3. **开启功能**：在门锁设置界面中，找到 **【靠近解锁远离上锁】** 功能选项，并**点击开启**该功能。

完成以上设置后，该功能即被激活。当您携带遥控钥匙靠近已锁止的车辆时，车辆会自动解锁；在所有车门关闭后，携带遥控钥匙远离车辆时，车辆会自动锁止。

【用户提问】
如何确保遥控钥匙的操作不受干扰？

【参考资料】
上车和下车
使用遥控钥匙解锁和闭锁 靠近解锁远离上锁功能
开启靠近解锁远离上锁功能，无需您取出遥控钥匙即可解锁和锁止车
辆。
■ 解锁：车辆处于锁止状态时，携带遥控钥匙靠近车辆时，车辆将
自动解锁。
■ 锁止：所有车门关闭后，携带遥控钥匙远离车辆时，车辆将自动
锁止。
在中央显示屏中点击 -车辆设置-门锁，进入门锁设置界面。
遥控钥匙
您的车辆配备了遥控钥匙。您可以通过遥控钥匙进行以下操作。
锁止按钮：短按锁止按钮，锁止所有车门/尾门，并启用防盗报
警。
解锁按钮：短按解锁按钮，解锁所有车门/尾门，并解除防盗报
警。
尾门按钮：短按尾门按钮，解锁尾门。长按尾门按钮，自动打开/
关闭尾门。尾门自动打开/关闭过程中，短按尾门按钮，尾门将停止 01点击开启/关闭靠近解锁远离上锁功能。
运动。
40
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何通过触摸车门外把手上车内解锁和闭锁的外部感应区域来操作车辆的闭锁？
【参考资料】
上车和下车
闭锁：关闭所有车门，携带遥控钥匙/蓝牙钥匙，触摸前门外把手上 车内解锁和闭锁
的外部感应区域，车辆闭锁。
注意！
■ 离车时，请确保带走遥控钥匙和蓝牙钥匙。
说明！
□ 按住车门把手外部传感器区域，关闭所有车窗和全景天窗。再次
触摸外部传感器，所有车窗和全景天窗关闭动作即可停止。
□ 遥控钥匙/蓝牙钥匙受到信号干扰时，无钥匙进入系统可能会功
能异常。
中控锁按键
按下驾驶员侧车门上的锁止按键，锁止所有车门和尾门。车辆锁
止时，按键上的指示灯会点亮。
按下驾驶员侧车门上的解锁按键，解锁所有车门和尾门。
44
【模型回答】
根据您提供的资料，操作步骤如下：

1. **确保条件满足**：您需要携带有效的遥控钥匙或蓝牙钥匙，并且所有车门都已关闭。
2. **执行操作**：用手触摸前门外把手上的**外部感应区域**，车辆即会执行闭锁操作，锁止所有车门和尾门。

补充说明：
* 根据资料，在闭锁状态下，按住外部感应区域还可以关闭所有车窗和全景天窗，再次触摸则可停止关闭动作。
* 请注意，如果遥控钥匙/蓝牙钥匙受到信号干扰，此无钥匙进入功能可能会出现异常。

以上信息来源于您提供的资料第44页。

【用户提问】
问题：如果遥控钥匙/蓝牙钥匙丢失或损坏，车辆如何进行闭锁和解锁操作？
【参考资料】
上车和下车
使用无钥匙进入系统解锁和闭锁
01设置无钥匙解锁（单门、全车）。
解锁：车辆闭锁状态下，携带遥控钥匙/蓝牙钥匙，触摸前门外把手
上的内部传感器区域，车辆解锁。
单门：当无钥匙解锁设置为单门时，触摸前门外把手上的内部传感器
区域仅解锁当前车门。
全车：当无钥匙解锁设置为全车时，触摸前门外把手上的内部传感器
区域解锁全车车门。
43
【模型回答】
结合给定的资料，无法回答问题。

【用户提问】
如何确保车辆在低速行驶或静止时，乘客安全地使用开门预警系统？
【参考资料】
上车和下车
检测区域
01点击开启/关闭主驾座椅迎宾功能。
DOW可检测对象：
■ 机动车辆。
说明！
■ 自行车。
□ 若驾驶员将座椅位置调整的很靠后时，主驾座椅迎宾功能将自动 ■ 电动车、摩托车。
停用。
■ 部分行人。
开门预警系统
当车辆静止或者行驶速度低于5km/h时，开门预警系统（DOW）针
对相邻两侧后方的交通状况进行检测。当开门存在潜在的碰撞风险
时，将闪烁外后视镜指示灯及氛围灯并发出声音警告，以提醒驾驶员
或乘客请勿开门以避免碰撞风险。
46

In [ ]: